Python libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classif
import seaborn as sns
import matplotlib.pyplot as plt
```

Load sample data

```
df = pd.read_csv('/content/sample_data/california_housing_train.csv')
display(df.head())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
0	-114.31	34.19	15.0	5612.0	1283.0
1	-114.47	34.40	19.0	7650.0	1901.0
2	-114.56	33.69	17.0	720.0	174.0
3	-114.57	33.64	14.0	1501.0	337.0
4	-114.57	33.57	20.0	1454.0	326.0

1. DataFrame Information

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
None
```

2. Descriptive Statistics of Numerical Columns

```
display(df.describe())
```

	longitude	latitude	housing_median_age	total_rooms	total_b
count	17000.000000	17000.000000	17000.000000	17000.000000	170(
mean	-119.562108	35.625225	28.589353	2643.664412	5(
std	2.005166	2.137340	12.586937	2179.947071	4(
min	-124.350000	32.540000	1.000000	2.000000	
25%	-121.790000	33.930000	18.000000	1462.000000	2(
50%	-118.490000	34.250000	29.000000	2127.000000	4(
75%	-118.000000	37.720000	37.000000	3151.250000	6(
max	-114.310000	41.950000	52.000000	37937.000000	64(

3. Handle Missing Values

First, let's check for any missing values.

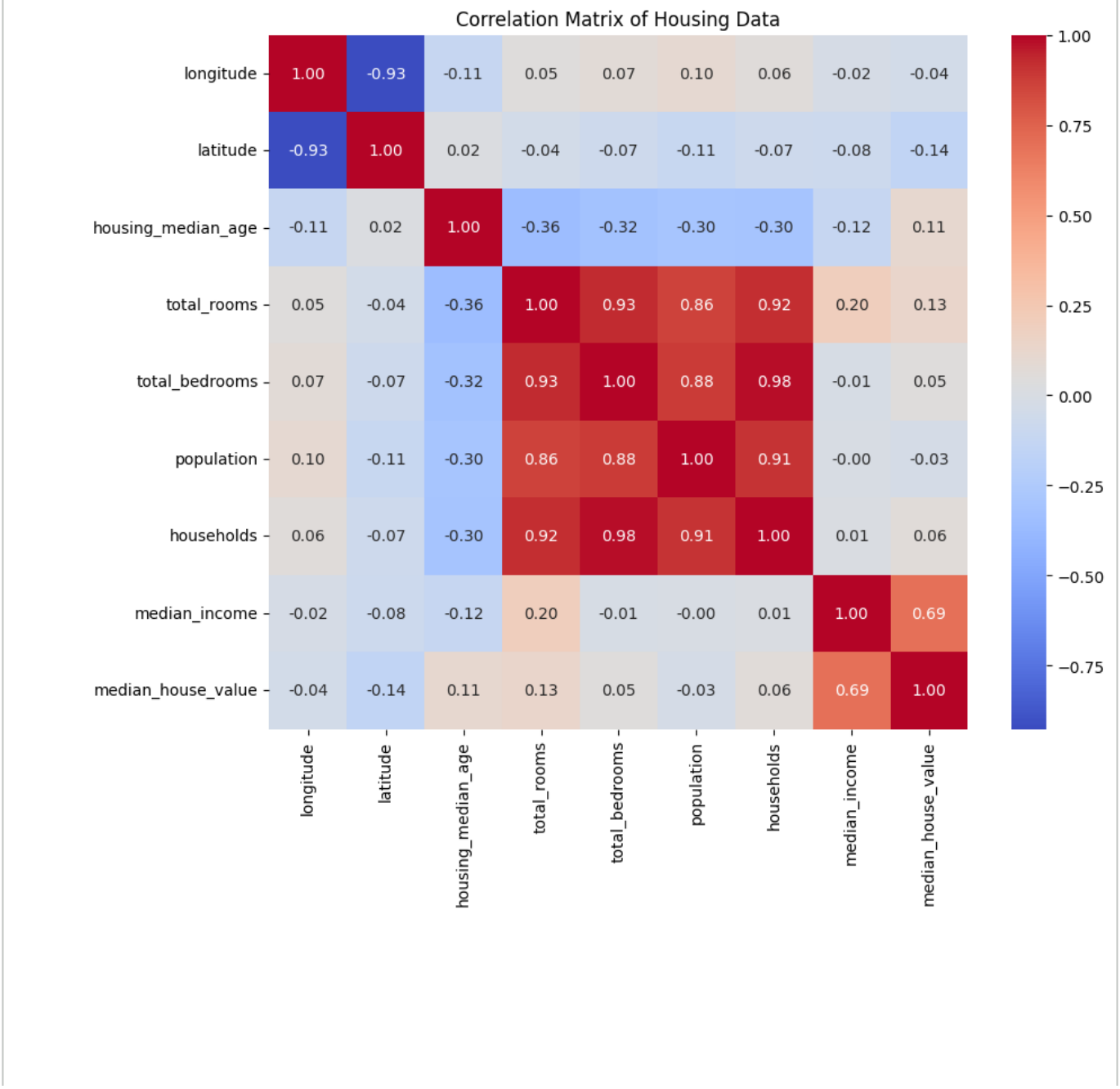
```
print(df.isnull().sum())

# It appears 'total_bedrooms' has some missing values. Let's impute tl
# The median is often preferred over the mean for imputation when data
if 'total_bedrooms' in df.columns:
    # Check again if there are actually missing values before attempti
    if df['total_bedrooms'].isnull().sum() > 0:
        median_bedrooms = df['total_bedrooms'].median()
        df['total_bedrooms'] = df['total_bedrooms'].fillna(median_bedr
        print("\nMissing values in 'total_bedrooms' imputed with media
    else:
        print("\nNo missing values found in 'total_bedrooms', imputati
    print(df.isnull().sum()) # Verify imputation
```

```
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
dtype: int64

No missing values found in 'total_bedrooms', imputation skipped.
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
dtype: int64
```

4. Identify Correlation

Let's visualize the correlation matrix to understand relationships between numerical features.

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Housing Data')
plt.show()
```



6. Create X and y variables for ML Model

We'll separate the features (`X`) from the target variable (`y`), which in this case is `median_house_value`.

```
# Define features (X) and target (y)
X = df.drop('median_house_value', axis=1)
y = df['median_house_value']

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

display(X.head())
display(y.head())
```

```
Shape of X: (17000, 8)
Shape of y: (17000,)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
0	-114.31	34.19	15.0	5612.0	1283.0
1	-114.47	34.40	19.0	7650.0	1901.0
2	-114.56	33.69	17.0	720.0	174.0
3	-114.57	33.64	14.0	1501.0	337.0
4	-114.57	33.57	20.0	1454.0	326.0

	median_house_value
0	66900.0
1	80100.0
2	85700.0
3	73400.0
4	65500.0

dtype: float64

7. Split the Data into Training and Testing Sets

To evaluate our machine learning model effectively, we'll split the data into training and testing sets. The training set will be used to train the model, and the testing set will be used to assess its performance on unseen data. A common split ratio is 80% for training and 20% for testing.

```
# Split the data into training and testing sets
# test_size=0.20 means 20% of the data will be used for testing
# random_state ensures reproducibility of the split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

```
Shape of X_train: (13600, 8)
Shape of X_test: (3400, 8)
Shape of y_train: (13600,)
Shape of y_test: (3400,)
```

8. Feature Scaling

Feature scaling is a crucial preprocessing step, especially for algorithms that are sensitive to the magnitude of features (e.g., gradient descent-based algorithms, distance-based algorithms like K-Nearest Neighbors, or Support Vector Machines). We will use `StandardScaler` to transform the features such that they have a mean of 0 and a standard deviation of 1. It's important to fit the scaler *only* on the training data to prevent data leakage from the test set.

```
# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform both training and
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Shape of X_train_scaled:", X_train_scaled.shape)
print("Shape of X_test_scaled:", X_test_scaled.shape)

# Display the first 5 rows of the scaled training features to show th
print("\nFirst 5 rows of scaled X_train:")
display(pd.DataFrame(X_train_scaled, columns=X_train.columns).head())
```

```
Shape of X_train_scaled: (13600, 8)
Shape of X_test_scaled: (3400, 8)

First 5 rows of scaled X_train:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
0	0.741830	-0.847157	-0.528398	0.033111	-0.445165
1	0.961406	-0.992433	-1.322210	0.451867	-0.008317
2	1.230885	-1.428262	-0.925304	-1.012397	-1.083819
3	1.195952	-1.315790	0.424176	-0.167966	-0.390559
4	0.741830	-0.678449	0.582938	-0.103400	-0.345450

9. Analyze Target Variable Distribution

Let's look at the normalized value counts of the target variable `y` to understand its distribution.

```
print("Normalized value counts for y:")
display(y.value_counts(normalize=True))
```

```
Normalized value counts for y:
```

median_house_value	proportion
500001.0	0.047882
137500.0	0.005588
162500.0	0.005235
112500.0	0.005000
187500.0	0.004353
...	...
442000.0	0.000059
328400.0	0.000059
296300.0	0.000059
307700.0	0.000059
441100.0	0.000059

3694 rows × 1 columns

dtype: float64

10. Convert Target Variable to Binary Classification

Since the request is to convert the `median_house_value` into a class variable (0 or 1), we will define a threshold. A common and robust choice is to use the median value of `median_house_value` as the threshold. This will create two roughly balanced classes:

- `0`: `median_house_value` is less than or equal to the median.
- `1`: `median_house_value` is greater than the median.

This conversion will be applied to the original `y` variable, as well as its training and testing subsets (`y_train` and `y_test`).

```
# Calculate the median of the original 'median_house_value'
median_house_value_threshold = y.median()
print(f"Median 'median_house_value' used as threshold: {median_house_v

# Convert y to a binary class variable
y_binary = (y > median_house_value_threshold).astype(int)
y_train_binary = (y_train > median_house_value_threshold).astype(int)
y_test_binary = (y_test > median_house_value_threshold).astype(int)

print("\nOriginal y head:\n")
display(y.head())
print("\nBinary y head:\n")
display(y_binary.head())

print("\nNormalized value counts for y_binary:")
display(y_binary.value_counts(normalize=True))
```

```
Median 'median_house_value' used as threshold: 180400.0

Original y head:
```

	median_house_value
0	66900.0
1	80100.0
2	85700.0
3	73400.0

4	65500.0

dtype: float64

```
Binary y head:
```

	median_house_value
0	0
1	0
2	0
3	0
4	0

dtype: int64

```
Normalized value counts for y_binary:
```

median_house_value	proportion
0	0.500471
1	0.499529

dtype: float64

11. Train a Random Forest Classification Model

Now that the target variable has been converted to a binary format and the features have been scaled, the next step is to train a classification model. A Random Forest Classifier is chosen for its robustness and good performance on various datasets. We will initialize it with specific parameters to control its complexity and prevent overfitting.

```
# Initialize the Random Forest Classifier with specified parameters
# n_estimators: The number of trees in the forest.
# max_depth: The maximum depth of the tree.
# min_samples_split: The minimum number of samples required to split a
# random_state: Controls the randomness of the bootstrapping of the sa
rf_classifier = RandomForestClassifier(n_estimators=100, max_depth=6,

# Train the classifier using the scaled training data and the binary t
rf_classifier.fit(X_train_scaled, y_train_binary)

print("Random Forest Classifier trained successfully.")
```

```
Random Forest Classifier trained successfully.
```

12. Evaluate the Random Forest Classifier

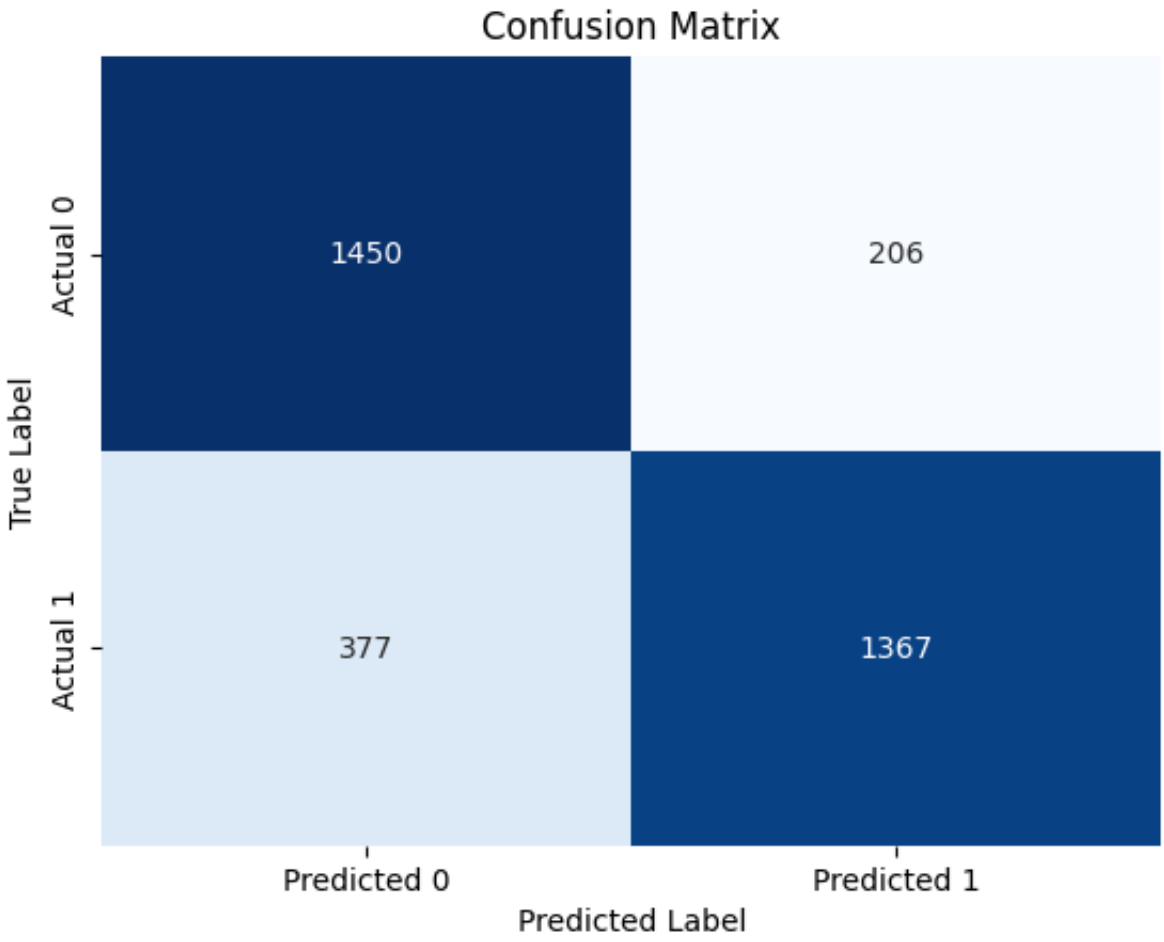
After training the model, it's crucial to evaluate its performance on the test set. We will use several metrics to assess how well our Random Forest Classifier is performing on the binary classification task. These metrics include:

- **Accuracy Score**: The proportion of correctly classified instances.
- **Classification Report**: Provides a detailed breakdown of precision, recall, f1-score, and support for each class.
- **Confusion Matrix**: A table showing the number of true positive, true negative, false positive, and false negative predictions.
- **ROC Curve and AUC**: The Receiver Operating Characteristic curve and the Area Under the Curve, which illustrate the classifier's performance across all possible classification thresholds.

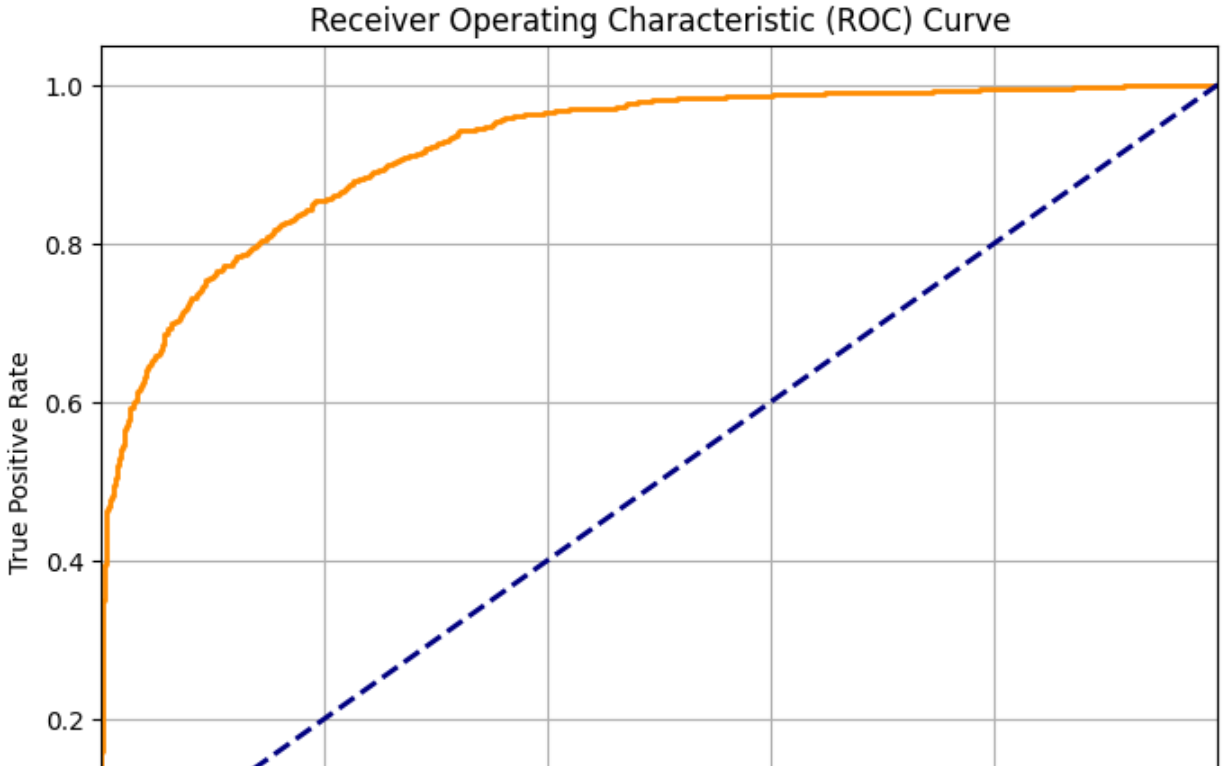
```
# Make predictions on the scaled test data
y_pred = rf_classifier.predict(X_test_scaled)

# Predict probabilities for the positive class (1) for ROC curve
y_pred_proba = rf_classifier.predict_proba(X_test_scaled)[:, 1]

# Evaluate the model
print("\nAccuracy Score:")
print(f"{accuracy_score(y_test_binary, y_pred):.4f}")

print("\nClassification Report:")
```

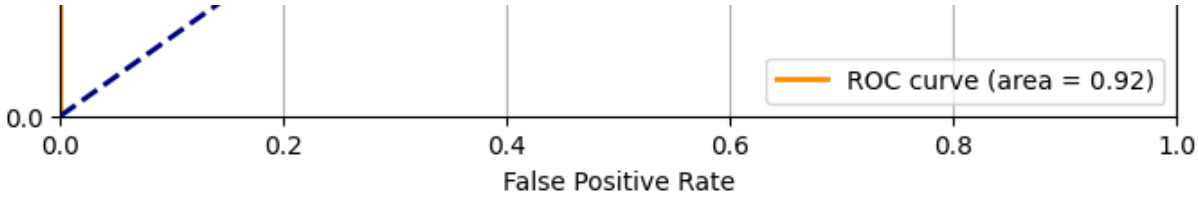
```
print(classification_report(y_test_binary, y_pred))

print("\nConfusion Matrix:")
cm = confusion_matrix(y_test_binary, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# Calculate ROC curve and AUC
fpr, tpr, thresholds = roc_curve(y_test_binary, y_pred_proba)
roc_auc = auc(fpr, tpr)

print(f"\nAUC Score: {roc_auc:.4f}")

# Plot ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area =
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

```
Accuracy Score:
0.8285

Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.88      0.83      1656
           1       0.87      0.78      0.82      1744

    accuracy                           0.83      3400
   macro avg       0.83      0.83      0.83      3400
weighted avg       0.83      0.83      0.83      3400


Confusion Matrix:
```




```
AUC Score: 0.9217
```





13. Perform Cross-Validation

Cross-validation is a technique to evaluate the machine learning models on a limited data sample. It helps in assessing the generalization ability of the model and provides a more reliable estimate of performance than a single train-test split. We will use 5-fold cross-validation on the training data.

```
# Perform 5-fold cross-validation
cv_scores = cross_val_score(rf_classifier, X_train_scaled, y_train_bi

print("Cross-validation scores:", cv_scores)
print(f"Mean cross-validation accuracy: {cv_scores.mean():.4f}")
print(f"Standard deviation of cross-validation accuracy: {cv_scores.s
```

```
Cross-validation scores: [0.82536765 0.82095588 0.83860294 0.83308824 (
Mean cross-validation accuracy: 0.8277
Standard deviation of cross-validation accuracy: 0.0071
```

14. Feature Importance Analysis

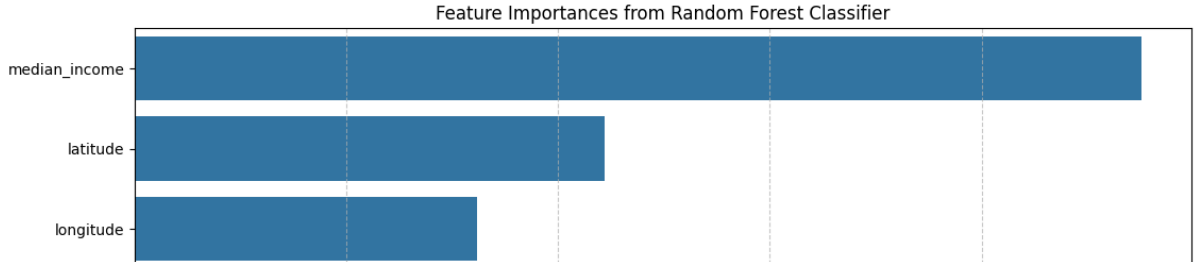
Understanding feature importance helps in interpreting the model and can provide insights into which variables are most influential in predicting the target. For tree-based models like Random Forest, we can extract feature importances directly from the trained model.

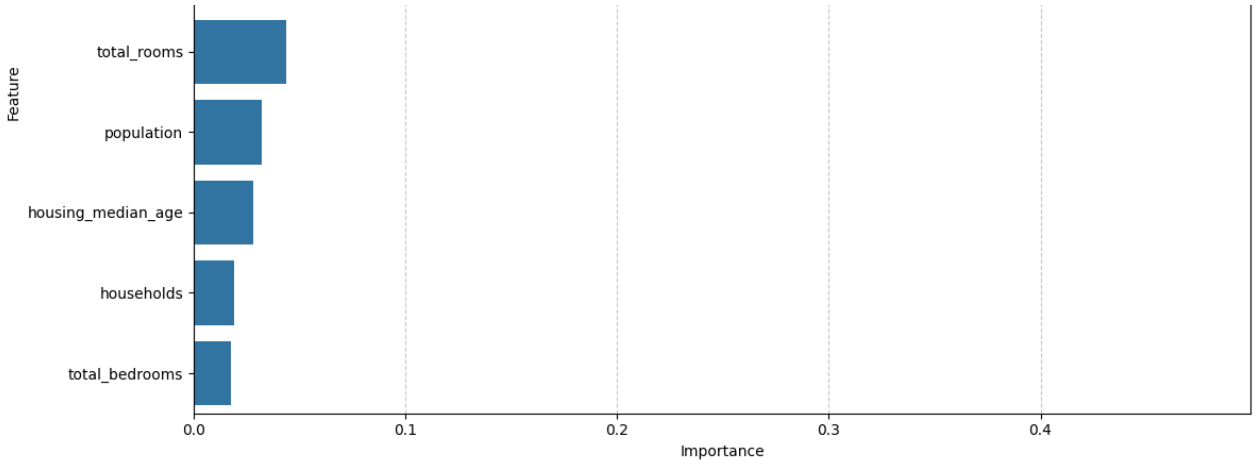
```
# Get feature importances from the trained Random Forest Classifier
feature_importances = rf_classifier.feature_importances_
```

```
# Get the feature names from the original DataFrame X
feature_names = X.columns

# Create a DataFrame to store feature names and their importances
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Impo

# Sort the DataFrame by importance in descending order
feature_importance_df = feature_importance_df.sort_values(by='Importa

print("\nFeature Importances:")
display(feature_importance_df)

# Optionally, visualize feature importances
plt.figure(figsize=(12, 7))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df)
plt.title('Feature Importances from Random Forest Classifier')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
Feature Importances:
```

	Feature	Importance
7	median_income	0.475409
1	latitude	0.222125
0	longitude	0.161652
3	total_rooms	0.043869
5	population	0.032330
2	housing_median_age	0.027945
6	households	0.019181
4	total_bedrooms	0.017489





Start coding or generate with AI.